

RAYA : Learning Where and When to Intervene for Robot Recovery

Anonymous Authors

Abstract—A robot can predict failure and still be unable to prevent it. By the time a safety mechanism reacts, the nominal plan may already have spent the control authority that recovery requires, and fixed task priorities may block whatever response remains. Our key insight is that both aspects are decided inside the controller. Recoverability must inform actions while they are chosen rather than veto them afterward, and task objectives must be adapted as recoverability shrinks. Building on this, we present **RAYA**, a hybrid learned–analytic framework that places a learned finite-horizon recoverability margin inside an optimal controller with hard constraints and pairs it with a bounded learned scheduler that shifts task weights to facilitate recovery. Across 7,200 simulation episodes per controller spanning quadrotor and autonomous-vehicle benchmarks, **RAYA** not only improves survival rates, but also transfers the learned components zero-shot to unseen trajectories, disturbances, plant shifts, and friction layouts. We developed an embedded realization of **RAYA** and deployed it onboard a 35g Crazyflie quadrotor. Across 40 combined hardware flights under wind with either aerodynamic mismatch or an unmodeled 40% motor-command loss, each of three baselines fails in all trials, while **RAYA** completes 10/10 six-cycle missions. Code and artifacts are open-sourced.

I. INTRODUCTION

Unexpected aerodynamic forces or a shifted payload may consume a quadrotor’s thrust and attitude authority [1]. Unmapped surfaces may similarly lack sufficient friction for a ground vehicle’s control authority [2]. Such failures are always preceded by a point past which no admissible input can enable recovery. Thus, knowing that a system is unsafe is not the same as being able to save it. It is important to consider *whether the controller can identify that recoverability is deteriorating while sufficient time and actuation authority remain*.

Common runtime safety architectures are often layered over the outputs of a nominal planning and control stack. Unfortunately, this means that when the safety layer changes the action, the plant may execute an input that leads to unplanned areas of the state space. Under tight input limits, such a safety mechanism may even be triggered too late to enable recovery. A rich family of safety filters implement this design, including control barrier function (CBF) quadratic programs [3], Hamilton–Jacobi (HJ) safety supervisors [4]–[7], and predictive safety filters [8], [9], alongside learned constructions that extend safety values to failure modes that are hard to model analytically [10]–[18].

Safety information can instead enter predictive optimization while an action is selected. Discrete-time CBF constraints and layered CBF–MPC architectures impose safety conditions inside the prediction horizon [19], [20]. Recent methods even embed learned neural barriers in sampling-based model predictive control (MPC) [21], [22], constrain

MPC with learned reachability-based safety values [23], or adapt predictive models online [24]. These approaches make risk visible, but they do not enable the planner and controller to reason about which task objectives can or should be modified in response. Thus, *recoverability often competes with task objectives during deployment*.

A third line of work addresses the task sacrifice directly. Context-dependent MPC weights [25], jointly adapted objective and barrier parameters [26], priority-based constraint relaxation [27], and recoverability-oriented barrier constructions [28] allow priorities to move at runtime, while reset policies and dedicated recovery controllers learn when to retreat, switch, or return to nominal operation [29]–[32]. These methods establish the value of adaptive priorities and of learning from failure experience, yet each resolves the question in one of two incomplete ways: either priorities adapt without a recoverability signal to drive them, or recovery is achieved through a discrete handoff to a separate action-generating policy. To our knowledge, no prior method jointly addresses *where* recoverability should enter the optimizer and *when* selected task objectives should yield, all within one planning and control framework.

We introduce **RAYA** (Recoverability-Aware Yielding of Authority), a hybrid learned–analytic control framework that uses a learned recoverability signal (Fig. 1). A neural model is trained on narrow platform-specific datasets and evaluated zero-shot on unseen trajectories, disturbance processes, plant shifts, and friction layouts. This model predicts the minimum task-specific safety margin over a finite closed-loop recovery horizon from signals available to the deployed controller, and a local approximation of that margin reshapes predictive feasibility while selecting the current action. The same margin also enters a lightweight bounded scheduler, together with deployable state and actuator-risk features, so objectives that can yield are temporarily downweighted while recovery-critical channels retain or gain weight. The prediction dynamics, state and input bounds, and platform-specific constraints remain unchanged. Thus, learning tracks when recovery is being lost and how strongly selected task objectives should yield, while the underlying optimal controller decides the final action subject to hard constraints.

Across 7,200 simulation episodes per controller spanning quadrotor and autonomous-vehicle (AV) benchmarks, **RAYA** not only improves survival rates, but also transfers the learned components zero-shot to unseen trajectories, disturbances, plant shifts, and friction layouts. We developed an embedded realization of **RAYA** and deployed it onboard a 35g Crazyflie quadrotor. Across 40 combined hardware flights under wind with either aerodynamic mismatch or

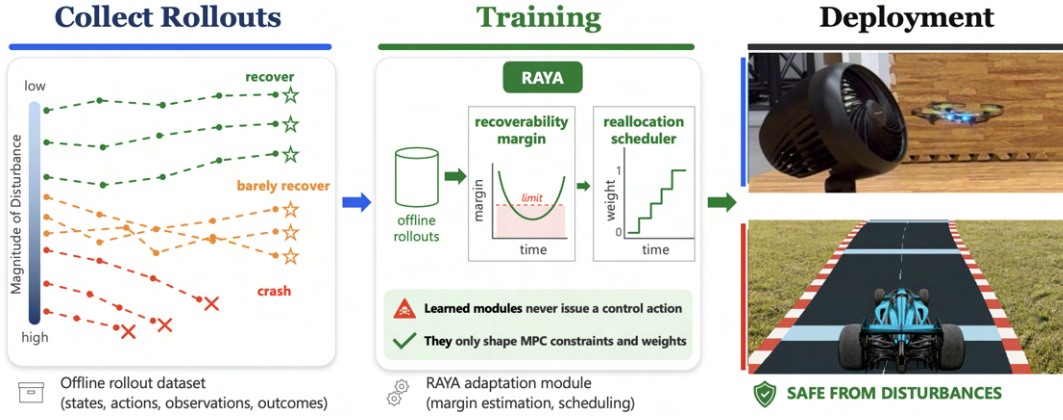


Fig. 1. **RAYA** learns when recoverability is deteriorating and how the task should respond. Closed-loop rollouts train a finite-horizon recoverability predictor and a bounded reallocation scheduler. At deployment, their outputs jointly modify a predictive control problem where the recoverability margin shapes the constraints, while the scheduler reweights selected tracking objectives before the control action is optimized.

an unmodeled 40% motor-command loss, each of three baselines fails in all trials, while **RAYA** completes 10/10 six-cycle missions. We release our code and artifacts open-source to benefit the wider robotics community.

Overall, this work makes three main contributions.

- **Hybrid Learned-Analytic Recovery.** A learned finite-horizon margin changes predictive feasibility, a bounded scheduler conditionally reweights selected task objectives, and the underlying optimal controller remains the action-generator subject to hard constraints.
- **Out-of-Distribution (OOD) Transfer.** Without retraining, the learned components generalize to unseen tasks, disturbance distributions, plant shifts, and friction layouts, and the same controller design transfers across aerial and ground embodiments.
- **Recovery without a Compute Bottleneck.** An embedded realization retains **RAYA**'s performance while running on an MCU on a 35g Crazyflie quadrotor.

II. BACKGROUND

At control time t , the controller receives a state estimate $\hat{x}_t$ of the state $x_t \in \mathcal{X} \subseteq \mathbb{R}^{n_x}$. Given reference states $x_{k|t}^{\text{ref}}$ and inputs $u_{k|t}^{\text{ref}}$, MPC solves

$$\min_{\mathbf{x}_t, \mathbf{u}_t} \sum_{k=0}^{N-1} \left(\|x_{k|t} - x_{k|t}^{\text{ref}}\|_{Q_0}^2 + \|u_{k|t} - u_{k|t}^{\text{ref}}\|_{R_0}^2 \right) + \|x_{N|t} - x_{N|t}^{\text{ref}}\|_{Q_f}^2 \quad (1a)$$

$$\text{s.t. } x_{0|t} = \hat{x}_t, \quad (1b)$$

$$x_{k+1|t} = A_{k|t}x_{k|t} + B_{k|t}u_{k|t} + c_{k|t}, \quad (1c)$$

$$x_{k|t} \in \mathcal{X}, \quad u_{k|t} \in \mathcal{U}, \quad k \in [0, N-1]. \quad (1d)$$

Here $A_{k|t}$, $B_{k|t}$, and $c_{k|t}$ define the local affine prediction model at stage k , N is the finite horizon, $\|a\|_Q^2 := a^\top Q a$, and Q_0 , R_0 , and Q_f are objective weights. The set $\mathcal{U} \subseteq \mathbb{R}^{n_u}$ denotes the admissible input set. We use $\mathbf{x}_t = \{x_{0|t}, \dots, x_{N|t}\}$ and $\mathbf{u}_t = \{u_{0|t}, \dots, u_{N-1|t}\}$. Only the first optimized input $u_{0|t}^*$ is applied before replanning.

For implementation, the problem can be expressed in the operator-splitting form

$$\min_{\xi, z} \frac{1}{2} \xi^\top H_t \xi + g_t^\top \xi + \Psi_t(z), \quad (2a)$$

$$\text{s.t. } E_t \xi = e_t, \quad (2b)$$

$$\xi - z = 0. \quad (2c)$$

Here ξ stacks the decision variables $\mathbf{x}_t$ and $\mathbf{u}_t$, z is the auxiliary variable used for separable projections, and $E_t \xi = e_t$ collects the initial-state and linearized dynamics constraints. The matrix H_t contains the quadratic objective weights induced by Q_0 , Q_f , and R_0 , while g_t contains the corresponding reference-dependent linear terms. The function Ψ_t collects the separable projections associated with the state and input constraints [33].

We solve (2) using the ADMM-based, Riccati-structured implementation of TinyMPC [34], [35]. For this work, the useful consequence of formulation (2) is that it separates two interfaces through which recovery information can modify the controller during action selection. Additional recoverability conditions enter through Ψ_t , while reweighting selected state-tracking objectives changes the corresponding objective terms in H_t and g_t . The prediction dynamics and pre-existing hard state and input constraints remain unchanged.

We distinguish post hoc from in-solver intervention. A post hoc safety filter modifies $u_{0|t}^*$ only after the nominal problem (1) has been solved. The predicted trajectory is therefore generated without accounting for that intervention, and a requested correction may be infeasible once the nominal plan has approached the input limits. An in-solver intervention instead modifies the optimization problem used to select $u_{0|t}^*$, allowing the predicted trajectory and current action to adapt jointly to task objectives and recoverability conditions. **RAYA** follows this architecture through both interfaces: recoverability modifies predictive feasibility, while objective reallocation changes which task objectives are prioritized. Section III develops these two interventions.

III. RAYA

Figure 1 summarizes the offline training and an online control step of **RAYA**. Offline, narrow, platform-specific, closed-loop rollouts provide worst-future-margin labels for training the learned recoverability predictor. Then, held-out rollouts calibrate its offset, and optimize a bounded objective reallocation scheduler in closed loop. Online, the learned predictor estimates the recoverability margin from the current state and recent history. Its local linearization enters the MPC problem of Sec. II as an additional constraint in Ψ_t . The same margin then feeds a bounded objective reallocation scheduler, $\bar{w}_t \in [0, 1]$, that reweights task objective channels, with the corresponding quadratic and reference-dependent objective terms, H_t, g_t , recomputed accordingly. The modified optimal control problem is solved once and $u_{0|t}^*$ is applied zero-shot to unseen scenarios. Neither learned component generates a control action, and the robot's dynamics, input bounds, and hard constraints remain unchanged.

A. Learning When Recovery Is Being Lost

Let $m : \mathcal{X} \rightarrow \mathbb{R}$ denote a signed instantaneous margin, with larger values indicating greater separation from failure. Given an offline closed-loop rollout, we label each time step by the minimum future margin over a recovery horizon H_{rec} ,

$$y_t = \text{smin}_{0 \leq j \leq H_{\text{rec}}} m_{t+j}, \quad (3)$$

where $m_t = m(x_t)$, j is the future-step offset, and smin denotes either the hard minimum or its smooth log-sum-exp approximation. We learn a regressor h_ψ that predicts this finite-horizon margin from deployable features ζ_t ,

$$y_t \approx h_\psi(\zeta_t) := f_\psi(\zeta_t) - T. \quad (4)$$

Here f_ψ is a neural network and T is calibrated on held-out closed-loop rollouts against false-safe predictions. A $\pm 50\%$ sweep of T changes at most one of 4,200 quadrotor outcomes and spans 1.5 points of AV survival. The features ζ_t use only deployable signals: altitude, vertical velocity, tilt, attitude margin, angular rate, and actuation authority for the quadrotor; tracking error, vehicle response, slip and model residuals, saturation, curvature, and short histories for the AV. Neither receives the realized disturbance or a preview.

B. Putting Recoverability Inside the Solver

The learned margin modifies the optimization problem with which $u_{0|t}^*$ is selected, rather than correcting the action post hoc. Let $\bar{x}_{k|t}$ denote the linearization point at prediction stage k , and let $\zeta_{k|t}(x_{k|t})$ denote the corresponding feature vector, with observed history quantities held fixed. We first form a first-order approximation of $x \mapsto h_\psi(\zeta_{k|t}(x))$ about $\bar{x}_{k|t}$,

$$\hat{h}_{k|t}(x) = h_\psi(\zeta_{k|t}(\bar{x}_{k|t})) + \nabla_x h_\psi(\zeta_{k|t}(\bar{x}_{k|t}))^\top (x - \bar{x}_{k|t}). \quad (5)$$

Defining $\tau_{b,k|t}$, the recoverability threshold at prediction stage k , the affine recoverability constraint can be imposed as follows,

$$\hat{h}_{k|t}(x_{k|t}) \geq \tau_{b,k|t}. \quad (6)$$

The threshold $\tau_{b,k|t}$ is separate from the calibration offset T . The offset T calibrates the learned recoverability prediction, while $\tau_{b,k|t}$ sets the margin enforced by MPC. Its platform-specific parameterization uses only quantities available at deployment, is fixed before evaluation, and is shared across the matched controller ablations.

When preserving feasibility of the underlying MPC problem is desired, we use a softened form with nonnegative slack $s_{k|t}$:

$$\hat{h}_{k|t}(x_{k|t}) + s_{k|t} \geq \tau_{b,k|t}, \quad s_{k|t} \geq 0, \quad (7)$$

and we add the penalty $\lambda_{k|t} s_{k|t}^2$, with $\lambda_{k|t} \geq 0$, to the nominal objective (1a).

The learned constraint therefore augments rather than replaces the pre-existing constraints already present in MPC. For example, for the quadrotor, we retain an altitude-descent guard and a minimum-collective constraint alongside the softened recoverability constraint (7), while for the AV, the learned recoverability constraint is combined with the existing trajectory-tube constraint.

Although h_ψ predicts a finite-horizon quantity, the corresponding constraint need not be imposed at every MPC stage. Let $\mathcal{K}_t$ denote the set of prediction stages at which the learned recoverability constraint is imposed. By default, we use only the first predicted stage, $\mathcal{K}_t = \{1\}$, in the selected controller. Sec. IV-B.3 compares the impact of this choice with deeper placements.

C. Learning When Task Priorities Should Yield

The recoverability constraint excludes or penalizes predicted states with low recoverability, but it does not specify when competing tracking objectives should yield as recoverability deteriorates. We therefore allow the reallocation scheduler to modify selected blocks of the nominal state-cost matrix Q_0 . Let $\mathcal{E}$ denote the yieldable task channels and $\mathcal{R}$ denote recovery-critical channels, those whose regulation becomes more important for recovery. We define

$$Q_{\mathcal{E}}(\bar{w}_t) = \max(\eta_{\min}, 1 - \beta_T \bar{w}_t) Q_{\mathcal{E}}^0, \quad (8)$$

$$Q_{\mathcal{R}}(\bar{w}_t) = (1 + \beta_A \bar{w}_t) Q_{\mathcal{R}}^0. \quad (9)$$

Here $0 < \eta_{\min} \leq 1$ is the minimum fraction of nominal weight retained on channels in $\mathcal{E}$, while $\beta_T \geq 0$ and $\beta_A \geq 0$ determine the amount of attenuation and amplification, respectively. All remaining entries of Q_0 are unchanged.

The choice of $\mathcal{E}$ and $\mathcal{R}$ depends on the control task. For the quadrotor, lateral position and velocity may yield while attitude and angular-rate regulation receive greater weight. For the AV, position and speed tracking may yield, while heading and steering regulation are preserved.

The scheduler is a smoothed logistic policy where $\phi_t \in \mathbb{R}^{11}$ contains a bias, a reactive margin ramp, the calibrated recoverability margin relative to its intervention threshold, deployable state and actuator-risk features drawn from ζ_t , and the previous scheduler output,

$$w_t = \sigma(\theta^\top \phi_t), \quad \bar{w}_t = \alpha \bar{w}_{t-1} + (1 - \alpha) w_t, \quad \alpha = 0.70 \quad (10)$$

We train the scheduler in closed loop using episodic cross-entropy search [36]. Candidate schedules are ranked by survival, then tracking error, then mean reallocation. The learned models and scheduler parameters are held fixed during all evaluations. The training distributions used for the experiments are described in Sec. IV-A.

D. Bounded-Intervention Guarantee

Having defined both learned components, we next state what their modifications can and cannot change in the underlying optimization. Although the learned recoverability margin is empirical, the way **RAYA** modifies MPC is structurally bounded. Let $\mathcal{C}_t$ denote the feasible set defined by the prediction dynamics and all pre-existing hard state, input, and platform constraints at time t . Since $\bar{w}_t \in [0, 1]$, $0 < \eta_{\min} \leq 1$, and $\beta_T, \beta_A \geq 0$, Eqs. (8)–(9) imply

$$\eta_{\min} Q_{\mathcal{E}}^0 \leq Q_{\mathcal{E}}(\bar{w}_t) \leq Q_{\mathcal{E}}^0, \quad Q_{\mathcal{R}}^0 \leq Q_{\mathcal{R}}(\bar{w}_t) \leq (1 + \beta_A) Q_{\mathcal{R}}^0. \quad (11)$$

Thus, with the positive-semidefinite state and terminal costs, $R_0 \succ 0$, and $\lambda_{k|t} \geq 0$, the **RAYA** objective remains convex for every scheduler output. Since scheduling modifies only the objective terms H_t and g_t , it leaves $\mathcal{C}_t$ unchanged and therefore cannot relax a pre-existing hard constraint. A hard learned recoverability constraint intersects $\mathcal{C}_t$ with an additional affine halfspace and therefore can only restrict the feasible set. For the softened recoverability constraint, every trajectory in $\mathcal{C}_t$ remains feasible by choosing

$$s_{k|t} = [\tau_{b,k|t} - \hat{h}_{k|t}(x_{k|t})]_+.$$

Hence the softened recoverability constraint cannot make an otherwise feasible MPC problem infeasible.

Thus, learning changes what the optimizer prefers and what it avoids, never what the platform is permitted to do.

IV. EXPERIMENTS

Our experiments map directly to our three contributions. We first evaluate the hybrid recovery mechanism by isolating the roles of in-solver integration and objective reallocation (Sec. IV-B.1). We then test whether **RAYA** expands the recoverable operating envelope beyond its training distribution (Sec. IV-B.2) and perform ablations to further isolate constraint depth, scheduling strategy, and component interaction (Sec. IV-B.3). Finally, we evaluate whether the same mechanism survives a performance-preserving embedded realization and physical deployment (Sec. IV-C).

A. Methodology

The quadrotor benchmark spans seven dimensionless disturbance multipliers from 6 to 12 across six evaluation conditions. On the Figure-8 disturbance profile in Fig. 2A, these produce 95th-percentile downward disturbance accelerations of 5.58–11.16 ms^{-2} . We report acceleration rather than airspeed because the simulator applies the disturbance directly as an acceleration without a calibrated aerodynamic mapping. The Figure-8 trajectory is used for training and Circle, Star, and Y-line trajectories, turbulent wind, and a plant with 20% greater mass and additional drag are held

out for zero-shot evaluation. We run 100 seeds at each disturbance level in each condition, giving 4,200 episodes per controller. An episode succeeds if the quadrotor completes all 281 control steps without violating the floor, attitude, or angular-rate limits.

The AV benchmark crosses five friction levels, $\mu \in \{0.20, 0.25, 0.30, 0.35, 0.40\}$, with three periodic-strip exposure durations, two random multi-patch counts, and one isolated single-patch condition. This gives 30 condition cells. Each condition cell contains 100 pre-generated geometry placements that are reused across all controllers, yielding 1,500 periodic-strip, 1,000 random multi-patch, and 500 single-patch episodes, or 3,000 episodes per controller. Only the periodic-strip family is included during training, while the random multi-patch and single-patch families are held out for zero-shot evaluation. Failure is defined as sustained departure from the 0.8 m trajectory tube, spinout, or severe slip, consistent with FITENTH safety-width scales [37].

In all cases, controllers do not receive true wind, friction, patch geometry, or future disturbance information unless explicitly identified as privileged. Learned models and scheduler parameters are fixed during evaluation and unless stated otherwise, all controllers are evaluated on identical disturbance seeds or geometry placements, and each ablation changes only the design choice being tested. In particular, post hoc versus in-solver comparisons use the same learned or HJ recoverability signal and change only where that signal enters the control pipeline. Baselines include nominal MPC, in-solver CBF and post hoc CBF filters [3], [19], reduced-order HJ supervisors [7], a sampling-based policy-CBF filter [15], and post hoc and in-solver uses of the same learned recoverability margin. All nine controllers appear in Fig. 2A. We call a condition empirically recoverable when at least 50% of its episodes succeed.

B. Simulation Experiments

1) *In-solver Integration and Objective Reallocation:* We first evaluate *where* the recoverability signal should enter the controller and *when* selected task objectives should yield. As shown in Fig. 2, on the quadrotor, nominal MPC achieves 34.24% survival, while a post hoc projection using the learned margin achieves 34.45% and is statistically indistinguishable from nominal control. Using the same learned margin inside MPC increases survival to 37.48%, an improvement of 3.02 percentage points over post hoc use (95% CI [+2.14, +3.90]). Adding reallocation scheduling further increases survival to 40.83%. Thus, on the quadrotor, both in-solver integration and objective reweighting contribute substantially to recovery. For reference, in-solver CBF only reaches 36.74%, below the in-solver learned margin.

A representative quadrotor trace shows the timing of the interventions. The learned recoverability signal activates at 0.75 s, and the scheduler output exceeds 0.5 at 0.85 s. At 0.85 s, the post hoc projection first becomes infeasible. Nominal and post hoc control fail at 5.45 s, while **RAYA** completes the 14.05 s mission. The hardware results in Sec. IV-C show the same pattern.

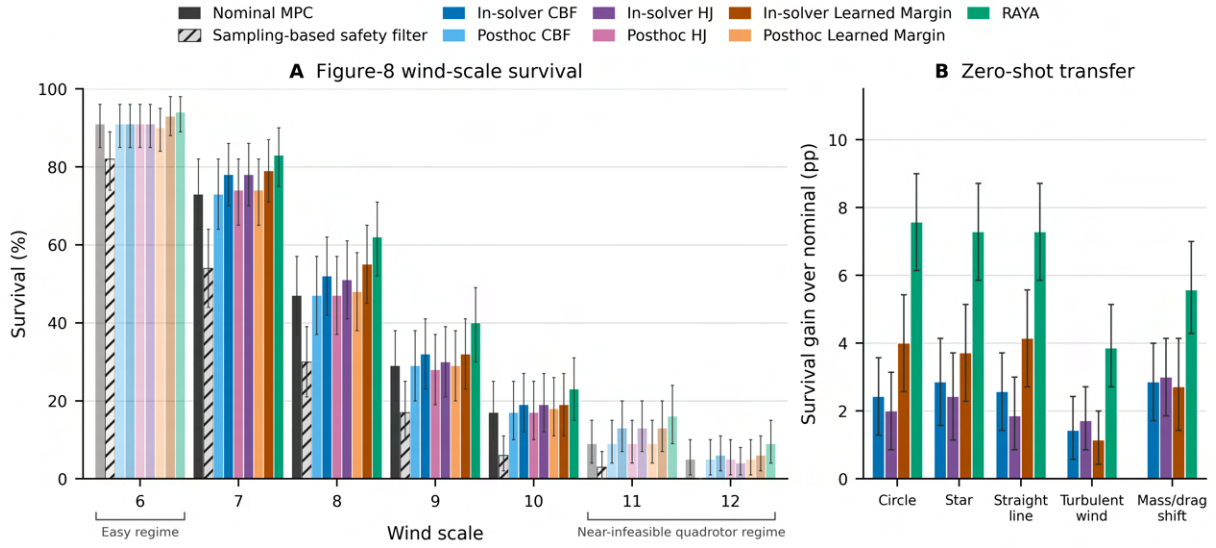


Fig. 2. **Quadrotor disturbance robustness and zero-shot transfer.** (A) Figure-8 survival across the wind sweep for nine controllers using 100 shared seeds per method–wind condition. Controller separation is largest between the benign and near-infeasible regimes. (B) Survival gain over nominal MPC for controllers that improve on unseen trajectories, turbulent wind, and mass/drag shift. Whiskers show 95% whole-seed clustered bootstrap intervals.

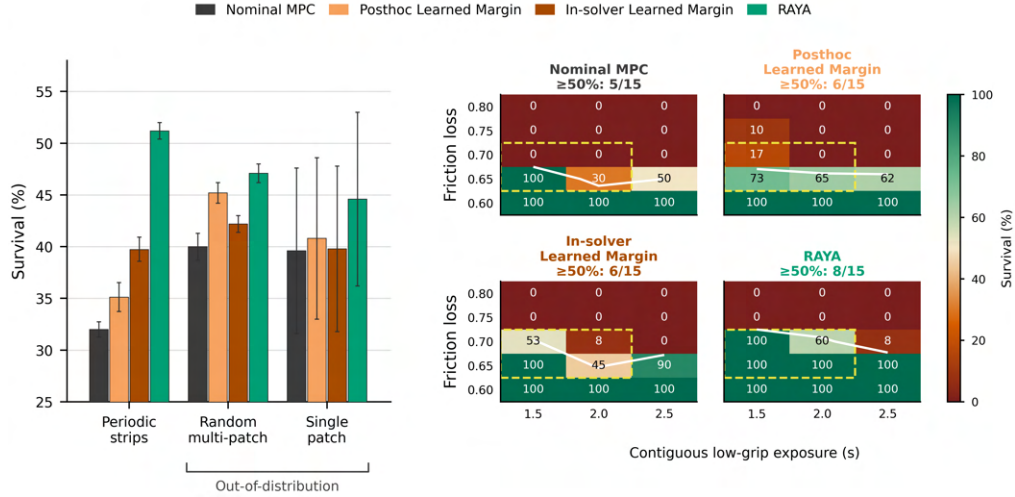


Fig. 3. **FITENTH recoverable operating envelope under latent friction loss.** Left. Survival on periodic strips and unseen random multi-patch and single-patch layouts. Whiskers show 95% placement-clustered bootstrap intervals. Right. Survival over the periodic-strip friction-loss-exposure grid using 100 shared placements per condition. White curves mark the 50%-survival boundary, and yellow boxes mark scenarios included during training.

We also find that using the same reduced-order HJ value as a post hoc filter gives 34.24% survival, whereas incorporating it directly into MPC gives 36.43%. Thus the benefit of in-solver integration is not specific to the learned margin, that is, the same signal is worth more inside the plan than after it, independent of how the signal is produced.

As shown in Fig. 3, AV results using an FITENTH [38] car show a different balance between the two components. Nominal MPC achieves 35.93% survival, a post hoc learned projection achieves 39.43%, and in-solver use of the learned margin reaches 40.57%. Adding reallocation scheduling raises survival to 48.73%. Here, in-solver integration provides a smaller improvement, while objective reweighting accounts for the larger gain. That is, the learned margin

identifies deteriorating recoverability, but effective recovery depends primarily on changing the tracking priorities before control authority is exhausted.

2) *The Recoverable Envelope Inside and Outside the Training Distribution:* Fig. 2A shows how the quadrotor benefit varies with disturbance severity. Under mild wind, all controllers perform similarly. At the strongest disturbances, survival collapses across controllers as the required correction approaches the available actuation limit. The largest separation occurs between these extremes, where recovery remains physically possible but requires earlier intervention. An offline analysis of the Figure-8 survival curves shows that RAYA shifts the interpolated 50%-survival threshold from 7.335 to 7.950 m s^{-2} of 95th-percentile injected downward

TABLE I

RECOVERABILITY-CONSTRAINT DEPTH ABLATION. ONLY K VARIES; LEARNED COMPONENTS ARE FIXED.

Platform	Configuration	Survival (95% CI)	Mean Reallocation
Quadrotor	Margin, $K = 1$	37.48 [33.36, 41.67]	0.000
Quadrotor	Margin, $K = 3$	36.33 [32.29, 40.57]	0.000
Quadrotor	RAYA , $K = 1$	40.83 [36.55, 45.24]	0.698
Quadrotor	RAYA , $K = 3$	39.98 [35.74, 44.36]	0.702
AV	RAYA , $K = 1$	48.73 [46.93, 50.53]	0.271
AV	RAYA , $K = 5$	48.70 [46.90, 50.50]	0.271

acceleration, an 8.4% increase, and raises the normalized survival-curve area from 37.17% to 45.92%. At disturbance multiplier 8, survival increases from 47% to 62%.

We next evaluate all nine controllers zero-shot on the held-out trajectory, wind, and plant families of Sec. IV-A. Only controllers improving over nominal MPC in every family are shown in Fig. 2B, and every controller that meets this criterion places recoverability inside the solver. **RAYA** shows an 18.9% relative gain across all held-out families, and under turbulent wind survival rises by up to 81.8% in the regime where nominal control begins to exhaust actuation authority.

The AV experiments test the same question under unobserved friction loss. Without retraining, **RAYA** achieves 51.2%, 47.1%, and 44.6% survival on the periodic, random multi-patch, and single-patch families, respectively, with the highest point estimate in each family (Fig. 3, left).

The friction-loss-exposure grid (Fig. 3, right) shows how **RAYA** expands the recoverable operating envelope. Under the 50%-survival criterion, nominal MPC is recoverable in 5/15 conditions, while the post hoc and in-solver learned-margin controllers are recoverable in 6/15 each. **RAYA** expands this to 8/15 conditions. Among the baselines, the in-solver learned margin sustains 50% survival to an interpolated exposure of 1.515 s. Nominal and post hoc control are already below 50% survival at the shortest tested exposure, so their thresholds lie below 1.5 s. **RAYA** extends the 50%-survival exposure to 2.096 s, a 38.4% increase over the strongest baseline.

3) *Ablation Studies:* Table I holds the learned models and scheduler fixed and varies only the number K of prediction stages at which the recoverability constraint is imposed. On the quadrotor, $K = 1$ performs slightly better than $K = 3$ for both the learned margin-only controller and **RAYA**. On the AV, $K = 1$ and $K = 5$ give nearly identical results. We therefore find no benefit from repeatedly imposing the learned constraint deeper into the MPC horizon.

Table II tests whether the gains come simply from using recovery-oriented weights vs. from changing them conditionally. As a fixed-weight baseline, we grid-search $\bar{w}$ on held-out validation rollouts and use the best selected value for each embodiment: $\bar{w} = 0.75$ for the quadrotor and $\bar{w} = 0.25$ for the AV. On the quadrotor, **RAYA** and always-on scheduling achieve nearly identical survival (40.83% and 40.88%), but **RAYA** uses 30.2% less mean reallocation and reduces XY tracking error by 27.5% on episodes completed by

TABLE II

REALLOCATION-SCHEDULING ABLATION. ONLY THE REALLOCATION SCHEDULE VARIES; THE LEARNED MARGIN AND REWEIGHTING RULE ARE FIXED. STATIC REALLOCATION IS SELECTED ON VALIDATION ROLLOUTS; LOWER MEAN REALLOCATION INDICATES LESS REWEIGHTING.

Quadrotor scheduler	Survival	XY error (m)	Mean Reallocation
No reallocation ($\bar{w} = 0$)	37.48	0.295	0.000
Handcrafted ramp	40.31	0.321	0.467
Validated static ($\bar{w} = 0.75$)	40.14	0.446	0.750
RAYA	40.83	0.390	0.698
Always-on ($\bar{w} = 1$)	40.88	0.537	1.000

AV Scheduler	Survival	Track error (m)	Mean Reallocation
No reallocation ($\bar{w} = 0$)	40.57	0.158	0.000
Handcrafted ramp	43.43	0.170	0.068
Validated static ($\bar{w} = 0.25$)	41.77	0.162	0.250
RAYA	48.73	0.161	0.271
Always-on ($\bar{w} = 1$)	44.77	0.207	1.000

both controllers. On the AV, conditional scheduling provides a larger survival benefit. **RAYA** reaches 48.73%, compared with 44.77% for always-on scheduling. At the same time **RAYA** uses mean reallocation 0.271 vs. 1.0 and reduces tracking error by 22.2%.

We further show the two mechanisms are not merely additive but also synergistic. On the quadrotor, scheduling without the learned margin reaches 34.83% survival, only 0.59 percentage points above nominal MPC. The in-solver learned margin alone reaches 37.48%, while combining both components reaches 40.83%, a 2.76-percentage-point gain beyond the sum of their individual improvements. On the AV, we compare against a privileged controller with a 300 ms preview of the true friction, which reaches 47.50% survival with mean reallocation 0.503. With no friction preview, **RAYA** reaches 48.73% survival while using 46.1% less mean reallocation (0.271 vs. 0.503).

C. Embedded Hardware Deployment

We deploy **RAYA** fully onboard the STM32F405 microcontroller (MCU) of a Crazyflie 2.1 Brushless [39] at 20 Hz using five ADMM iterations and a 50 ms control deadline. Each control step adds one recoverability-network evaluation and gradient, one scheduler evaluation, one learned recoverability constraint, and updates to selected tracking weights. Each controller is tested in five flights under each of the two conditions, aerodynamic mismatch and unmodeled motor loss, under the same two-fan airflow (see Fig. 4). A mission is successful if all six Figure-8 cycles are completed and the vehicle lands under control. All controllers retain their nominal model and constraints and receive no measurement or model of the disturbance or mismatch.

Table III shows a clear separation between the controllers. **RAYA** completes all five flights in both conditions, totaling 30/30 commanded cycles per condition. Nominal MPC, in-solver CBF, and the post hoc learned controller do not complete a single six-cycle mission in either condition. In one motor-loss flight, **RAYA** contacts the floor after 3.286

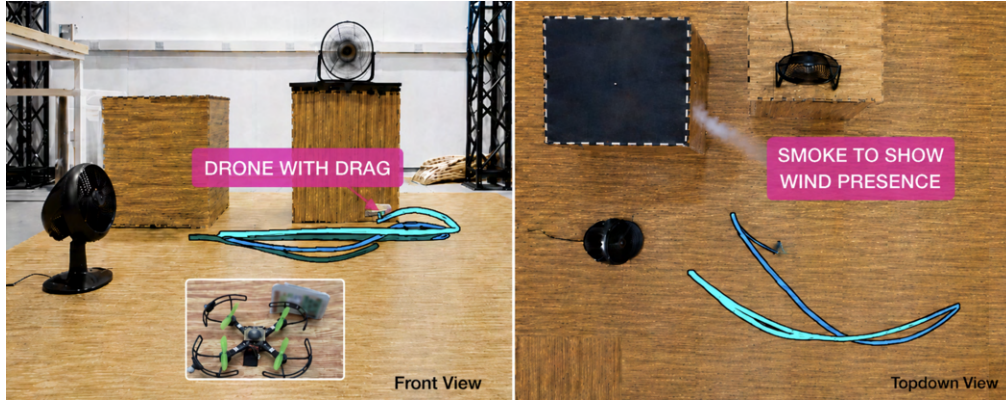


Fig. 4. **Six-cycle onboard recovery testbed.** Both conditions use the same Figure-8 mission under two-fan airflow. The aerodynamic-mismatch condition adds a 3 g attachment, while the motor-loss condition applies an unmodeled 40% reduction to the front-left motor command.

TABLE III

REAL-WORLD PERFORMANCE OVER FIVE FLIGHTS PER CONDITION.
BOTH CONDITIONS USE THE SAME SIX-CYCLE FIGURE-8 MISSION
UNDER TWO-FAN AIRFLOW. REPORTED CYCLE COUNTS AND FAILURE
TIMES ARE AVERAGED OVER FIVE FLIGHTS.

Method	Mission complete	Mean cycles completed	Mean time to failure (s)
3 g attachment + wind			
Nominal MPC	✗ 0/5	0.615/6	2.44
In-solver CBF	✗ 0/5	1.188/6	4.74
Post hoc Learned Margin	✗ 0/5	2.135/6	8.55
RAYA	✓ 5/5	6.000/6	–
40% M1 loss + wind			
Nominal MPC	✗ 0/5	1.326/6	5.32
In-solver CBF	✗ 0/5	3.702/6	14.85
Post hoc Learned Margin	✗ 0/5	2.022/6	8.11
RAYA	✓ 5/5	6.000/6	–

cycles, recovers, completes the remaining trajectory, and lands under control. Mission completion under motor loss is therefore 5/5, with 4/5 flights completed without contact.

We also verify that the embedded implementation preserves the behavior of the full controller. The recoverability network and its gradient are evaluated in single precision, and a static 64-entry Riccati cache replaces online recomputation of the corresponding feedback quantities. Across more than one million replayed states, single-precision inference produces no changes in recoverability activation. Cache quantization changes feedback gains by at most 1.18%, and a paired 4,200-episode simulation audit changes only 27 paired outcomes, yielding 40.86% survival versus 40.83% for continuous precision. The network and cache together occupy 161 KiB of the MCU’s 1 MiB flash.

All controllers also meet the 50 ms onboard deadline. Under aerodynamic mismatch, the worst observed control-step latency is 30.40 ms for nominal MPC, 31.55 ms for in-solver CBF, 42.95 ms for the post hoc learned controller, and 44.32 ms for **RAYA**. Under motor loss, the corresponding values are 29.35, 33.79, 42.46, and 44.52 ms. The post hoc learned controller therefore already uses 85–86% of the available deadline, compared with approximately 89% for

RAYA, yet completes none of the ten hardware missions while **RAYA** completes all ten.

The failure modes are also consistent with the simulation results. The post hoc projection is infeasible for 93.2% of attempted corrections under aerodynamic mismatch and 97.9% under motor loss. In-solver CBF becomes active but still fails in every flight. Together, the replay and timing audits show that neither numerical conversion nor additional computation explains **RAYA**’s hardware advantage. By the time the post hoc filter requests a correction, there is almost never an admissible input left to deliver it. The difference is whether recoverability shapes the plan early enough, and whether the task is allowed to yield.

V. CONCLUSION AND FUTURE WORK

We introduced **RAYA**, built on a simple premise. Recoverability should not be checked only after the action is chosen. It should shape the plan, and what the task is allowed to sacrifice, while recovery is still possible. Across aerial and ground embodiments, **RAYA** improves recovery over post hoc intervention, expands the recoverable operating envelope, and transfers without retraining to unseen tasks, disturbances, plant shifts, and friction layouts. The same design also transfers to embedded hardware, where **RAYA** maintains recovery under aerodynamic mismatch and unmodeled actuator degradation.

Several questions remain. The recoverability features and recovery-critical objective channels are chosen by the designer, so while the optimizer interface transfers across embodiments, some engineering remains. A natural step is to learn which features predict deteriorating recoverability and which objectives should yield. This matters most for learned nominal policies, contact-rich manipulation, and legged locomotion, where failure precursors are high-dimensional, history-dependent, and hard to enumerate in advance and we are excited to extend **RAYA** to these domains.

VI. ACKNOWLEDGMENTS

We used LLM tools, including Codex and Claude, to assist with software implementation and proofreading. All final text and references were edited and reviewed by humans.

REFERENCES

- [1] D. Hanover, P. Foehn, S. Sun, E. Kaufmann, and D. Scaramuzza, "Performance, precision, and payloads: Adaptive nonlinear mpc for quadrotors," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 690–697, 2021.
- [2] S. Vaskov, R. Quirynen, M. Menner, and K. Berntorp, "Friction-adaptive stochastic nonlinear model predictive control for autonomous vehicles," *Vehicle System Dynamics*, vol. 62, no. 2, pp. 347–371, 2024.
- [3] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2016.
- [4] J. F. Fisac, A. K. Akametalu, M. N. Zeilinger, S. Kaynama, J. Gillula, and C. J. Tomlin, "A general safety framework for learning-based control in uncertain robotic systems," *IEEE Transactions on Automatic Control*, vol. 64, no. 7, pp. 2737–2752, 2018.
- [5] S. Bansal, M. Chen, S. Herbert, and C. J. Tomlin, "Hamilton-jacobi reachability: A brief overview and recent advances," in *2017 IEEE 56th annual conference on decision and control (CDC)*. IEEE, 2017, pp. 2242–2253.
- [6] S. Bansal and C. J. Tomlin, "Deepreach: A deep learning approach to high-dimensional reachability," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 1817–1824.
- [7] J. Borquez, K. Chakraborty, H. Wang, and S. Bansal, "On safety and liveness filtering using hamilton-jacobi reachability analysis," *IEEE Transactions on Robotics*, vol. 40, pp. 4235–4251, 2024.
- [8] K. P. Wabersich and M. N. Zeilinger, "A predictive safety filter for learning-based control of constrained nonlinear dynamical systems," *Automatica*, vol. 129, p. 109597, 2021.
- [9] B. Tearle, K. P. Wabersich, A. Carron, and M. N. Zeilinger, "A predictive safety filter for learning-based racing control," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 7635–7642, 2021.
- [10] A. Robey, H. Hu, L. Lindemann, H. Zhang, D. V. Dimarogonas, S. Tu, and N. Matni, "Learning control barrier functions from expert demonstrations," in *2020 59th IEEE Conference on Decision and Control (CDC)*. IEEE, 2020, pp. 3717–3724.
- [11] L. Lindemann, H. Hu, A. Robey, H. Zhang, D. Dimarogonas, S. Tu, and N. Matni, "Learning hybrid control barrier functions from data," in *Conference on robot learning*. PMLR, 2021, pp. 1351–1370.
- [12] H. Ma, B. Zhang, M. Tomizuka, and K. Sreenath, "Learning differentiable safety-critical control using control barrier functions for generalization to novel environments," in *2022 European Control Conference (ECC)*. IEEE, 2022, pp. 1301–1308.
- [13] L. Lindemann, A. Robey, L. Jiang, S. Das, S. Tu, and N. Matni, "Learning robust output control barrier functions from safe expert demonstrations," *IEEE Open Journal of Control Systems*, vol. 3, pp. 158–172, 2024.
- [14] O. So, Z. Serlin, M. Mann, J. Gonzales, K. Rutledge, N. Roy, and C. Fan, "How to train your neural control barrier function: Learning safety filters for complex input-constrained systems," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 11 532–11 539.
- [15] L. Knoedler, O. So, J. Yin, M. Black, Z. Serlin, P. Tsiotras, J. Alonso-Mora, and C. Fan, "Safety on the fly: Constructing robust safety filters via policy control barrier functions at runtime," *IEEE Robotics and Automation Letters*, 2025.
- [16] E. Daş, D. E. van Wijk, T. G. Molnar, A. D. Ames, and J. W. Burdick, "Robust adaptive backup control barrier functions," *arXiv preprint arXiv:2607.20842*, 2026.
- [17] A. Lin, S. Peng, and S. Bansal, "One filter to deploy them all: Robust safety for quadrupedal navigation in unknown environments," *IEEE Transactions on Robotics*, vol. 42, pp. 545–560, 2025.
- [18] C. Dawson, S. Gao, and C. Fan, "Safe control with learned certificates: A survey of neural lyapunov, barrier, and contraction methods for robotics and control," *IEEE Transactions on Robotics*, vol. 39, no. 3, pp. 1749–1767, 2023.
- [19] J. Zeng, B. Zhang, and K. Sreenath, "Safety-critical model predictive control with discrete-time control barrier function," in *2021 American control conference (ACC)*. IEEE, 2021, pp. 3882–3889.
- [20] R. Grandia, A. J. Taylor, A. D. Ames, and M. Hutter, "Multi-layered safety for legged robots via control barrier functions and model predictive control," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 8352–8358.
- [21] J. Yin, O. So, E. Y. Yu, C. Fan, and P. Tsiotras, "Safe beyond the horizon: Efficient sampling-based mpc with neural control barrier functions," *arXiv preprint arXiv:2502.15006*, 2025.
- [22] A. U. Kaypak, P. Krishnamurthy, and F. Khorrami, "A control barrier function-constrained model predictive control framework for safe reinforcement learning," *arXiv preprint arXiv:2604.06463*, 2026.
- [23] H. Wang, N. Nguyen, A. Jordana, L. Righetti, and S. Bansal, "Coop-timizing safety and performance using safety value-constrained model predictive control," *arXiv preprint arXiv:2604.23863*, 2026.
- [24] H. Zhou and V. Tzoumas, "Simultaneous system identification and model predictive control with no dynamic regret," *IEEE Transactions on Robotics*, vol. 41, pp. 4322–4341, 2025.
- [25] B. Zarrouki, M. Spanakakis, and J. Betz, "A safe reinforcement learning driven weights-varying model predictive control for autonomous vehicle motion control," in *2024 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2024, pp. 1401–1408.
- [26] E. Sabouni, H. S. Ahmad, V. Giammarino, C. G. Cassandras, I. C. Paschalidis, and W. Li, "Reinforcement learning-based receding horizon control using adaptive control barrier functions for safety-critical systems," in *2024 IEEE 63rd Conference on Decision and Control (CDC)*. IEEE, 2024, pp. 401–406.
- [27] F. Prignoli, Y. S. Quan, M. Jeddli, J. Sjöberg, and P. Falcone, "Priority-driven safe model predictive control approach to autonomous driving applications," *arXiv preprint arXiv:2505.05933*, 2025.
- [28] M. Wijayatunga, N. Wallace, S. Sukkarieh, and R. Armellin, "Learning safety-guaranteed, non-greedy control barrier functions using reinforcement learning," in *2026 IEEE Aerospace Conference*. IEEE, 2026, pp. 1–14.
- [29] B. Eysenbach, S. Gu, J. Ibarz, and S. Levine, "Leave no trace: Learning to reset for safe and autonomous reinforcement learning," *arXiv preprint arXiv:1711.06782*, 2017.
- [30] B. Thananjeyan, A. Balakrishna, U. Rosolia, F. Li, R. McAllister, J. E. Gonzalez, S. Levine, F. Borrelli, and K. Goldberg, "Safety augmented value estimation from demonstrations (saved): Safe deep model-based rl for sparse cost robotic tasks," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3612–3619, 2020.
- [31] B. Thananjeyan, A. Balakrishna, S. Nair, M. Luo, K. Srinivasan, M. Hwang, J. E. Gonzalez, J. Ibarz, C. Finn, and K. Goldberg, "Recovery rl: Safe reinforcement learning with learned recovery zones," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4915–4922, 2021.
- [32] S. Vats, D. K. Jha, M. Likhachev, O. Kroemer, and D. Romeres, "Recoverychaining: Learning local recovery policies for robust manipulation," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 9776–9783.
- [33] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Foundations and Trends® in Machine Learning*, vol. 3, no. 1, pp. 1–122, 2011.
- [34] K. Nguyen, S. Schoedel, A. Alavilli, B. Plancher, and Z. Manchester, "Tinympc: Model-predictive control on resource-constrained microcontrollers," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [35] I. Mahajan, K. Nguyen, S. Schoedel, E. Nedumaran, M. Mata, B. Plancher, and Z. Manchester, "Code generation and conic constraints for model-predictive control on microcontrollers with conic-tinympc," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [36] R. Y. Rubinstein and D. P. Kroese, *The cross-entropy method: a unified approach to combinatorial optimization, Monte-Carlo simulation, and machine learning*. Springer, 2004, vol. 133.
- [37] N. Baumann, E. Ghignone, J. Kühne, N. Bastuck, J. Becker, N. Imholz, T. Kränzlin, T. Y. Lim, M. Lötscher, L. Schwarzenbach, et al., "Forzaeth race stack—scaled autonomous head-to-head racing on fully commercial off-the-shelf hardware," *Journal of Field Robotics*, vol. 42, no. 4, pp. 1037–1079, 2025.
- [38] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "Fltenth: An open-source evaluation environment for continuous control and reinforcement learning," *Proceedings of Machine Learning Research*, vol. 123, 2020.
- [39] W. Giernacki, M. Skwarczyński, W. Witwicki, P. Wroński, and P. Kozierski, "Crazyflie 2.0 quadrotor as a platform for research and education in robotics and control engineering," in *2017 22nd international conference on methods and models in automation and robotics (MMAR)*. IEEE, 2017, pp. 37–42.